

SYSTEM ARCHITECTURE & DESIGN SPECIFICATION

Automotive Cluster GUI Project

Detailed Design & Deployment Specification on BeagleBone Black (Qt 6.11.1)

Hardware Platform: BeagleBone Black
(Cortex-A8, 512MB RAM)

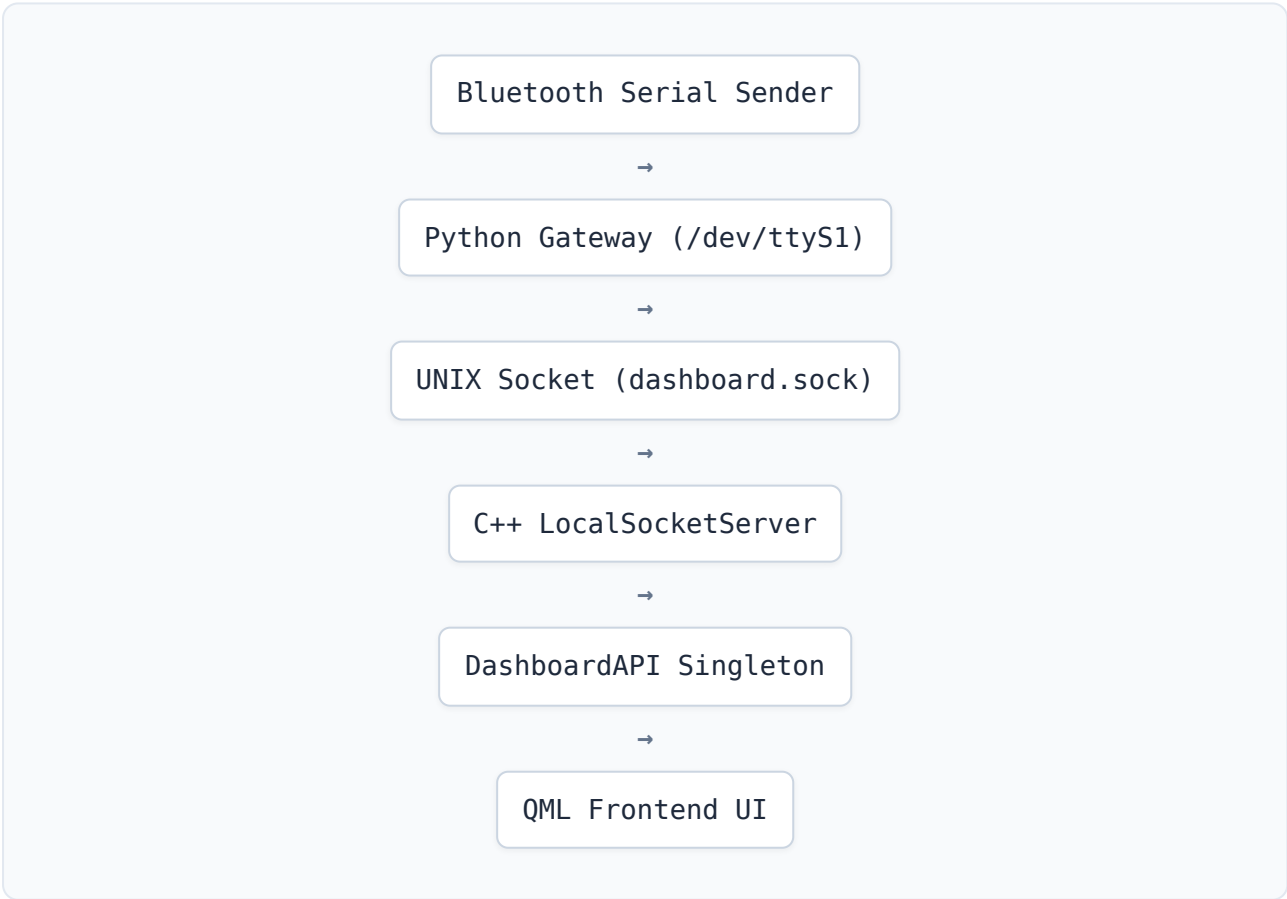
Software Engine: Qt 6.11.1 / QML (VNC
Server backend)

Date: June 25, 2026

Status: Approved & Implemented

1. System Overview & Architecture

This design document details the technical specification of the Automotive Cluster GUI project. The cluster application is optimized for low-latency visual updates and operates on resource-constrained target hardware by dividing work into independent, decoupled processes communicating via a local UNIX domain socket.



Subsystem Descriptions:

- **Serial-to-Socket Gateway (`bt_gateway.py`)**: Formulates line-buffered incoming serial payloads from the Bluetooth device into formatted command pairs, delivering them to the local UNIX domain socket.
- **Local Socket Server (`LocalSocketServer`)**: Running as a C++ service thread inside the GUI application, it listens to the UNIX domain socket, resolves escape characters, splits bulk payloads, and updates the core API.
- **Dashboard API (`DashboardAPI`)**: Serves as the central C++ singleton property sheet, exposing vehicle states to QML via declarative property bindings and emitting change notifications.
- **QML Presentation Layer**: Visually renders gauges, warnings, and overlays, and scales elements dynamically to fit the hardware's rendering viewport.

2. Target Hardware Environment & Performance Optimization

The target platform is a **BeagleBone Black (BBB)** Single Board Computer running a headless Debian Linux distribution. Due to the lack of hardware graphics acceleration and limited system resources, visual operations require strict constraints to avoid CPU throttling and power crashes:

USB Current Brownout Mitigation

QML rendering on a single-core Cortex-A8 (512MB RAM) without GPU support can easily saturate the CPU, spiking current draw. If powered via USB, this results in immediate board reboots. The design resolves this by optimizing render size and capping frame delivery rates.

- **Resolution Scaling:** The application window is bounded at `800x480` to minimize framebuffer drawing sizes. The layout itself is structured on a canvas of `1024x600` and scaled down dynamically via a QML `Scale` transform.
- **VNC Draw-Rate Control:** The system is started with VNC platform parameters: `- platform vnc:size=800x480:draw-rate=10`. Limiting VNC updates to 10 FPS reduces CPU load to 20-30%, ensuring hardware stability.

3. Backend Integration Layer (C++)

The C++ backend bridges incoming socket raw byte streams to the reactive QML GUI bindings.

3.1. DashboardAPI Property Sheet

The `DashboardAPI.h` and `DashboardAPI.cpp` files define a singleton context property:

- **Properties Exposed:**
 - *Sensors:* `throttle` (double), `battery` (double), `temperature` (double).
 - *Gears:* `gear` (QString: P, R, N, D).
 - *Warnings:* `tirePressureLow`, `seatbeltWarning`, `cruiseControlActive`, `electricalFault`, `absWarning`.
 - *Indicators:* `leftIndicator`, `rightIndicator`, `brakePressed`, `highBeam`.
 - *System:* `bootActive` (bool).
- **Optimization:** Fuzzy floating-point comparison (`qFuzzyCompare`) prevents emitting signals for redundant data modifications, reducing property-update processing overhead.

3.2. LocalSocketServer UNIX Domain Socket Receiver

The socket server class `LocalSocketServer.h` and its implementation `LocalSocketServer.cpp` process incoming commands:

- **Multi-Line Escape Parsing:** The server reads incoming stream buffers, parses escaped characters (replacing literal text escapes like `\\n` and `\\r` with true Control Codes), and separates the commands based on newline boundaries.
- **Command Dispatches:** Splits keys and values matching `key:value`, converting states into appropriate boolean, integer, or string commands before calling corresponding C++ API slots.

4. Frontend Presentation Layer (Qt/QML)

The user interface renders the dashboard cluster using responsive, modular QML components.

4.1. Layout Scaling Matrix

In `Main.qml`, the `Window` container scales the elements according to the system aspect ratios:

```
property real scaleFactor: Math.min(width / 1024.0, height / 600.0)
transform: Scale {
    origin.x: 0; origin.y: 0
    xScale: root.scaleFactor
    yScale: root.scaleFactor
}
```

4.2. UI Components

- `ArcGauge.qml`: Draws circular indicators for battery and throttle.
 - *Blink Animation*: Employs a completed startup transition that mimics an eyeball blink once the boot sequence finishes.
 - *Behavior Easing*: Implements `Behavior` on `displayedValue` with a `900ms` transition timer. This animation is skipped (set to `0`) during the boot sweep for precise manual timer coordination.
 - *Low Power Warning*: Flashes red with a `300ms` timer once the battery level falls below 20%.
- `CarStatus.qml`: Integrates vector asset overlays on top of `car_blueprint.svg`. Displays door state indicators (rotating red panels), brake lights (red highlights), headlights (`headlight_beam.svg`), blinkers (500ms flash cycles), and warning icon badges.
- `GearSelector.qml`: Renders selector buttons for P, R, N, D, highlighting the current transmission state in cyan.
- `TopHeader.qml`: Updates datetime information. Below 3°C, it automatically triggers a snowy layout (exchanging weather indicators to `weather_snow.svg` with blue

temperature fonts).

4.3. Animated Boot Sweep

Upon launch, the interface coordinates a manual 60 FPS sweep sequence:

1. **Up-Sweep:** Sweeps speed and battery indicators from 0 to 100.
2. **Down-Sweep:** Sweeps speed back to 0, holding battery indicators at 100.
3. **Setup Release:** Releases warning indicators and marks `bootActive` as false, enabling live socket inputs.

5. Data Communications & Python Gateway

Incoming Bluetooth serial commands are forwarded to the C++ server via `bt_gateway.py`:

- **Raw Serial Hook:** Uses standard library `termios` to configure `/dev/ttyS1` at `9600` baud in raw mode (8 data bits, no parity, 1 stop bit, no flow control).
- **Socket Connection Handshake:** Retries connection to the C++ UNIX domain socket at ``/home/debian/dashboard.sock`` every 2 seconds if a connection drop is encountered.

6. Orchestration & Deployment

Compilation and deployment systems configure directories, stop services, copy files, and configure system dependencies:

- **CMake Setup:** `CMakeLists.txt` links core components and embeds SVGs/QML sources directly inside resources (``qrc``) to package them inside the binary.
- **Service Control:** To deploy new builds, the SSH script `run_unified_deploy.py` stops background daemons to unlock target executables. It then copies the files, runs `setup_board.sh` to apply permissions, and restarts the services.
- **Systemd Units:**
 - `automotive-dashboard.service` : Configures system runtime libraries paths (pointing to local `/home/debian/qt6.11`) and runs the UI at VNC resolution and limited draw rate.
 - `bt-serial-gateway.service` : Executes and maintains the Python serial gateway process.